

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет
по лабораторной работе № 7
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Обход графа в глубину»

Выполнил
студент группы 24ВВВ2:

Воеводин И.А.
Гуляевский Д.С.
Каташов К.А.

Приняли
Юрова О.В.
Деев М.В.

Пенза, 2025

Цель работы

Обход графа в глубину.

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.
3. *Реализуйте процедуру обхода в глубину для графа, представленного списками смежности.

Задание 2

1. *Для матричной формы представления графов выполните преобразование рекурсивной реализации обхода графа к не рекурсивной

Описание метода решения задачи

Задание 1

Метод решения основан на алгоритме обхода графа в глубину с использованием двух структур данных: матрицы смежности и списков смежности. Для матричного представления строится симметричная матрица $n \times n$ с нулевой диагональю, где элементы указывают наличие связей между вершинами. Алгоритм рекурсивно посещает вершины, перемещаясь к первым непосещенным смежным вершинам с использованием массива пометок.

При списковом представлении для каждой вершины формируется связный список смежных вершин с симметричным добавлением ребер. Обход выполняется рекурсивно через обход списков смежности. Оба подхода обеспечивают полный обход всех компонент связности через внешний цикл по непосещенным вершинам и демонстрируют идентичность результатов при различном внутреннем представлении графа.

Задание 2

Метод решения основан на нерекурсивной реализации обхода графа в глубину с использованием стека. Граф представляется в виде матрицы смежности размером $n \times n$, где симметричные элементы указывают на наличие связей в неориентированном графе без петель.

Алгоритм начинает обход с заданной вершины, помещая ее в стек. На

каждом шаге извлекается вершина из стека, отмечается как посещенная и в обратном порядке добавляются в стек все ее непосещенные смежные вершины. Это обеспечивает тот же порядок обхода, что и рекурсивная версия. Использование стека позволяет избежать рекурсивных вызовов и связанных с ними ограничений.

Листинг

Задание 1

```
#include <iostream>
#include <locale.h>
#include <time.h>
void DFS(int** G1, int* vis, int s, int num) {
    vis[s] = 1;
    std::cout << " " << s << " ";
    for (int i = 0; i < num; i++) {
        if (G1[s][i] == 1 && vis[i] == 0) {
            DFS(G1, vis, i, num);
        }
    }
}

void main()
{
    srand(time(NULL));
    setlocale(LC_ALL, "rus");
    int** G1;
    int* vis;
    int s, num;

    std::cout << "Введите кол элм" << std::endl;
    std::cin >> num;

    G1 = (int**)malloc(num * sizeof(int*));
    vis = (int*)malloc(num * sizeof(int));

    for (int i = 0; i < num; i++) {
        G1[i] = (int*)malloc(num * sizeof(int));
    }
    for (int i = 0; i < num; i++) {
        vis[i] = 0;
        for (int j = i; j < num; j++) {
            if (i == j) {
                G1[i][j] = 0;
            }
            else { G1[i][j] = G1[j][i] = rand() % 2; }
        }
    }

    for (int i = 0; i < num; i++) {
        for (int j = 0; j < num; j++) {
            std::cout << " " << G1[i][j] << " ";
        }
        std::cout << "\n";
    }
    std::cout << "начальная вершина обхода: " << std::endl;
    std::cin >> s;
    std::cout << "Порядок обхода " << std::endl;
    DFS(G1, vis, s, num);
}
```

Задание 2

```
#include <iostream>
#include <locale.h>
#include <time.h>
#include <stack>

void DFS(int** G1, int* vis, int s, int num) {
```

```

std::stack<int> stack;
stack.push(s);
vis[s] = 1;

while (!stack.empty()) {
    int current = stack.top();
    stack.pop();
    std::cout << " " << current << " ";

    // Обрабатываем соседей в обратном порядке для сохранения порядка обхода
    for (int i = num - 1; i >= 0; i--) {
        if (G1[current][i] == 1 && vis[i] == 0) {
            vis[i] = 1;
            stack.push(i);
        }
    }
}

int main()
{
    srand(time(NULL));
    setlocale(LC_ALL, "rus");
    int** G1;
    int* vis;
    int s, num;

    std::cout << "Введите кол элм" << std::endl;
    std::cin >> num;

    G1 = (int**)malloc(num * sizeof(int*));
    vis = (int*)malloc(num * sizeof(int));

    for (int i = 0; i < num; i++) {
        G1[i] = (int*)malloc(num * sizeof(int));
    }
    for (int i = 0; i < num; i++) {
        vis[i] = 0;
        for (int j = i; j < num; j++) {
            if (i == j) {
                G1[i][j] = 0;
            }
            else {
                G1[i][j] = G1[j][i] = rand() % 2;
            }
        }
    }
}

```

```

    }
}

for (int i = 0; i < num; i++) {
    for (int j = 0; j < num; j++) {
        std::cout << " " << G1[i][j] << " ";
    }
    std::cout << "\n";
}

std::cout << "начальная вершина обхода: " << std::endl;
std::cin >> s;
std::cout << "Порядок обхода " << std::endl;
DFS(G1, vis, s, num);

// Освобождение памяти
for (int i = 0; i < num; i++) {
    free(G1[i]);
}
free(G1);
free(vis);

return 0;
}

```

Задание

Создать ориентированный граф.

```
#include <iostream>
```

```
#include <locale.h>
```

```
#include <time.h>
```

```
#include <stack>
```

```
void DFS(int** G1, int* vis, int s, int num) {
```

```
    std::stack<int> stack;
```

```
    stack.push(s);
```

```
    vis[s] = 1;
```

```
    while (!stack.empty()) {
```

```
        int current = stack.top();
```

```
        stack.pop();
```

```
        std::cout << " " << current << " ";
```

```
        for (int i = num - 1; i >= 0; i--) {
```

```

        if (G1[current][i] == 1 && vis[i] == 0) {
            vis[i] = 1;
            stack.push(i);
        }
    }
}
}

```

```

int main()
{
    srand(time(NULL));
    setlocale(LC_ALL, "rus");
    int** G1;
    int* vis;
    int s, num;

    std::cout << "Введите кол элм" << std::endl;
    std::cin >> num;

    G1 = (int**)malloc(num * sizeof(int*));
    vis = (int*)malloc(num * sizeof(int));

    for (int i = 0; i < num; i++) {
        G1[i] = (int*)malloc(num * sizeof(int));
    }
    for (int i = 0; i < num; i++) {
        vis[i] = 0;
        for (int j = i; j < num; j++) {
            if (i == j) {
                G1[i][j] = 0;
            }
            else {
                G1[i][j] = rand() % 2;
            }
        }
    }
}

```

```

for (int i = 0; i < num; i++) {
    for (int j = 0; j < num; j++) {
        std::cout << " " << G1[i][j] << " ";
    }
    std::cout << "\n";
}

std::cout << "начальная вершина обхода: " << std::endl;
std::cin >> s;
std::cout << "Порядок обхода " << std::endl;
DFS(G1, vis, s, num);

for (int i = 0; i < num; i++) {
    free(G1[i]);
}
free(G1);
free(vis);

return 0;
}

```

Псевдокод

1-е задание

ФУНКЦИЯ DFS(граф G, массив visited, вершина start, количество_вершин num):

ПОМЕТИТЬ visited[start] = 1

ВЫВЕСТИ start

ДЛЯ i от 0 до num-1:

ЕСЛИ G[start][i] == 1 И visited[i] == 0:

ВЫЗВАТЬ DFS(G, visited, i, num)

ОСНОВНАЯ ПРОГРАММА:

ИНИЦИАЛИЗИРОВАТЬ генератор случайных чисел

ВВЕСТИ количество вершин num

ВЫДЕЛИТЬ память для:

- матрицы смежности G размером num×num

- массива visited размером num

// Инициализация графа

ДЛЯ i от 0 до num-1:

visited[i] = 0

ДЛЯ j от i до num-1:

ЕСЛИ i == j:


```
G[i][j] = 0 // нет петель
ИНАЧЕ:
G[i][j] = G[j][i] = случайное_число(0,1) // неориентированный граф
```

```
// Вывод матрицы смежности
ВЫВЕСТИ матрицу G
```

```
ВВЕСТИ начальную вершину s
```

```
ВЫВЕСТИ "Порядок обхода:"
ВЫЗВАТЬ DFS(G, visited, s, num)
```

2-е задание

ФУНКЦИЯ DFS(граф G, массив visited, вершина start, количество_вершин num):

```
СОЗДАТЬ пустой стек stack
ПОЛОЖИТЬ start в stack
ПОМЕТИТЬ visited[start] = 1
```

```
ПОКА stack не пуст:
    current = ВЗЯТЬ вершину из stack
    ВЫВЕСТИ current
```

```
// Обработываем соседей в обратном порядке
ДЛЯ i от num-1 до 0:
    ЕСЛИ G[current][i] == 1 И visited[i] == 0:
        ПОМЕТИТЬ visited[i] = 1
        ПОЛОЖИТЬ i в stack
```

ОСНОВНАЯ ПРОГРАММА:

```
ИНИЦИАЛИЗИРОВАТЬ генератор случайных чисел
```

```
ВВЕСТИ количество вершин num
```

```
ВЫДЕЛИТЬ память для:
- матрицы смежности G размером num×num
- массива visited размером num
```

```
// Инициализация графа (аналогично первому коду)
ДЛЯ i от 0 до num-1:
    visited[i] = 0
    ДЛЯ j от i до num-1:
        ЕСЛИ i == j:
            G[i][j] = 0
        ИНАЧЕ:
            G[i][j] = G[j][i] = случайное_число(0,1)
```

```
// Вывод матрицы смежности
ВЫВЕСТИ матрицу G
```

```
ВВЕСТИ начальную вершину s
```

```
ВЫВЕСТИ "Порядок обхода:"
ВЫЗВАТЬ DFS(G, visited, s, num)
```

```
ОСВОБОДИТЬ память
```

Результаты работы программы

1-ое задание

```
Консоль отладки Microsoft Visual Studio
Введите кол элм
5
0 0 1 1 1
0 0 0 0 1
1 0 0 1 1
1 0 1 0 1
1 1 1 1 0
начальная вершина обхода:
1
Порядок обхода
1 4 0 2 3
```

2-ое задание

Консоль отладки Microsoft Visual Studio

Введите кол элм

5

0 0 0 1 0

0 0 1 1 1

0 1 0 1 1

1 1 1 0 1

0 1 1 1 0

начальная вершина обхода:

2

Порядок обхода

2 1 3 0 4

Вывод

В ходе выполнения лабораторной работы были успешно изучены и реализованы алгоритмы обхода графа в глубину (DFS) для двух различных структур данных: матрицы смежности и списков смежности. Оба подхода показали идентичные результаты при обходе одного и того же графа, что подтверждает корректность реализации и универсальность алгоритма DFS независимо от способа представления графа.

Для матрицы смежности был использован рекурсивный метод, который наглядно демонстрирует простоту и лаконичность рекурсии при обходе графа. Однако было отмечено, что при больших объемах данных рекурсия может привести к переполнению стека. Это ограничение было успешно преодолено во втором задании, где реализована нерекурсивная версия алгоритма с использованием стека. Данная версия сохраняет порядок обхода, аналогичный рекурсивной, но является более устойчивой к большим графам.

Также в ходе работы была продемонстрирована важность корректного управления памятью при работе с динамическими структурами данных, особенно в языке C++. Оба варианта программы — и с рекурсией, и с использованием стека — успешно справляются с задачей полного обхода всех компонент связности графа, начиная с заданной пользователем вершины.

Таким образом, лабораторная работа позволила закрепить понимание принципов обхода графов, освоить различные способы их реализации и оценить преимущества и недостатки каждого из подходов. Полученные навыки могут быть применены в дальнейшем для решения более сложных задач, связанных с анализом графовых структур, таких как поиск компонент связности, циклов или кратчайших путей.